

Exceptions — Խորը ուղեցույց

C++ Exception Handling ամբողջական presentation

Այս ուղեցույցը մանրամասն բացատրում է C++ exception-ների try/catch/throw, stack unwinding, exception safety, RAII-ի հետ կապ, noexcept, custom exception, օրինակներով և interview հարցերով.

1. Ի՞նչ է Exception-ը

Exception-ի **skzbunqayin sxali (error) mshakman mekanizm** է. Erb sxal e patahum, kod-y "throw" e anum exception, vory "catch" e linum ayl texum -- sarqum e normal hosqy u error-neri handling-y arancin.

```
#include <stdexcept>

int divide(int a, int b) {
    if (b == 0)
        throw std::runtime_error("Division by zero"); // throw
    return a / b;
}

int main() {
    try {
        divide(10, 0);
    } catch (const std::runtime_error& e) { // catch
        cout << "Error: " << e.what();
    }
}
```

2. try / catch / throw

```
try {  
    // kod vor karox e throw anel  
    throw std::runtime_error("problem");  
}  
catch (const std::runtime_error& e) {    // konkret tip  
    cout << e.what();  
}  
catch (const std::exception& e) {        // base tip (bolor standard)  
    cout << e.what();  
}  
catch (...) {                            // amen inch (catch-all)  
    cout << "unknown error";  
}
```

```
throw  -> exception nety  
try    -> "pahpanvac" seksia  
catch  -> exception brnum, tesakov entrum (verevic var)
```

3. Stack Unwinding (amenakarevory)

Erb throw e linum, C++-y "het e gnum" (unwind) call stack-ov minchev gtni hamapatashan catch. **Ayd entacqum bolor local object-neri destructor-nery kanchvum en** -> RAII cleanup.

```
void f() {  
    std::string s = "local";           // RAII object  
    std::vector<int> v(100);           // RAII object  
    throw std::runtime_error("x");    // throw  
    // s u v-i destructor-nery AVTOMAT kanchvum (stack unwinding)  
}
```

Dra hamar RAII-y exception-safe e. throw-i zham resource-nery avtomat azatvum en.

4. Exception-neri hierarchia (std::exception)

```
std::exception (base, .what())
+-- std::logic_error
|   +-- std::invalid_argument
|   +-- std::out_of_range      (vector::at, etc.)
|   +-- std::length_error
+-- std::runtime_error
    +-- std::overflow_error
    +-- std::range_error
std::bad_alloc      (new anhajoxutyun)
std::bad_cast       (dynamic_cast reference)
```

```
try {
    std::vector<int> v(3);
    v.at(10);           // throw std::out_of_range
} catch (const std::out_of_range& e) {
    cout << e.what();
}
```

5. Custom Exception

```
class MyException : public std::exception {
    std::string msg;
public:
    MyException(const std::string& m) : msg(m) {}
    const char* what() const noexcept override { // override what()
        return msg.c_str();
    }
};

throw MyException("custom error");
```

Sovoraky `ja'angel` `std::exception` `-ic`, `override` `anel` `what()`.

6. Exception Safety Guarantees

Function-neri exception safety-i 4 makardak.

1. No-throw (noexcept) -> erbek chi throw (amenaxist)
2. Strong -> throw-i depqum vichak-y chi poxvum (rollback, "all or nothing")
3. Basic -> throw-i depqum invariant pahvum (chka leak), bayc vichak karox poxvum
4. No guarantee -> karox e leak / corruption (VAT)

RAII-y sovorabar apahovum e arnvazn **basic** guarantee (chka leak).

7. noexcept

```
void safe() noexcept {           // xostanum e erbek chi throw
    // ...
}
```

```
void safe() noexcept {
    throw 1;    // noexcept function throw anum -> std::terminate (crash)
}
```

noexcept -y. (1) compiler optimization, (2) move constructor-neri noexcept linelu depqum STL-y ogtagorcum e move (ne copy). Ete noexcept function throw e anum -> **std::terminate**.

8. Karevor kanonner

- catch by reference (const std::exception&), NE by value (slicing)
- Destructor-ic chthrow (throw dtor-ic -> std::terminate)
- RAII ogtagorci -> exception-safe cleanup
- catch konkretic -> general (out_of_range afaj, exception heto)
- Mi ogtagorci exception normal control flow-i hamar (danak e)

catch by reference (ne value)

```
catch (std::exception e)           // SXAL -- slicing, copy
catch (const std::exception& e)    // CHISHT -- reference, polymorphic
```

9. Interview harcer & patasxanner

Exception inch e?

Error-neri mshakman mekhanizm; throw/try/catch; normal hosq u error handling arancin.

Stack unwinding inch e?

throw-i zham C++-y het e gnum call stack-ov catch gtnelu; ayd entacqum local object-neri destructor-ner kanchvum (RAII cleanup).

Inchu catch by reference?

By value -> slicing (derived exception-y ktrvum) + copy. Reference -> polymorphic, chka slicing.

Destructor-ic kareli e throw anel?

Voch -- stack unwinding-i zham erkrord throw -> std::terminate. dtor petq e noexcept.

noexcept inch e?

Xostum vor function chi throw; compiler optimization + STL move ogtagorcum. Throw anelis -> std::terminate.

Exception safety guarantees?

No-throw, strong (rollback), basic (chka leak), no guarantee. RAII -> arnvazn basic.

std::exception hierarchia?

Base `std::exception` (.what()); `logic_error` (`invalid_argument`, `out_of_range`), `runtime_error` (`overflow`), `bad_alloc`, `bad_cast`.

RAII-i het kapy?

RAII apahovum e exception-safe cleanup (stack unwinding -> dtor -> resource azatum).

Cheat Sheet

EXCEPTION = error handling: throw / try / catch

```
throw std::runtime_error("msg");  
try { ... } catch (const std::exception& e) { e.what(); } catch(...) {}
```

STACK UNWINDING: throw -> het call stack-ov -> local dtor-ner kanchvum (RAII cleanup)

HIERARCHY: `std::exception` -> `logic_error` (`out_of_range`) / `runtime_error` / `bad_alloc`

SAFETY: `no-throw(noexcept)` > `strong(rollback)` > `basic(no leak)` > `none`

KANONNER:

```
catch by REFERENCE (const&) ne value (slicing)  
dtor-ic CHTHROW (std::terminate)  
noexcept -> chi throw (aylapes terminate) + optimization  
RAII -> exception-safe cleanup  
custom: jařangel std::exception, override what()
```